

# Gaming Rank & Latency Analyzer

Author: Tejushree

Repository: gaming\_rank\_analyzer

Status: 100% Complete (Tiers 1–4)



## Executive Summary

The **Gaming Rank & Latency Analyzer** is an end-to-end data engineering and statistical analysis project designed to assess server performance (ping latency), player matchmaking stats, and match outcome distributions across global gaming regions (**APAC, LATAM, EU**).

The primary objective was to evaluate whether server latency differs significantly by geographical region, which could impact competitive balance, player satisfaction, and overall matchmaking fairness.



## Complete Technical Stack & Dependencies

### Python Libraries Used

- `sqlite3`: Integrated Python standard library used to create, connect to, and execute SQL queries on the relational database ( `gaming_data.db` ).
- `pandas`: Used for data manipulation, loading raw CSVs, cleaning missing values, joining relational datasets, and querying SQLite directly into DataFrames ( `pd.read_sql_query` ).
- `matplotlib.pyplot`: Primary plotting engine used to format canvas dimensions, clear visual layouts, and save generated charts directly to high-resolution PNG assets.
- `seaborn`: Advanced statistical visualization library built on Matplotlib. Used `sns.boxplot` for ping distribution plots and `sns.countplot` for regional match outcome breakdowns.
- `scipy.stats`: Advanced statistical computing module. Used the `f_oneway` function to conduct a parametric One-Way Analysis of Variance (ANOVA) test.

### Tools & Infrastructure

- VS Code & Jupyter Extension**: Local Integrated Development Environment (IDE) running `wrangling.ipynb`.
- Python Virtual Environment ( `.venv` )**: Isolated environment managing local package dependencies.
- Git & GitHub**: Version control system used to track changes, tag commits ( `feat:` , `docs:` ), and host the final repository online.



## Step-by-Step Breakdown Across All 4 Tiers

### Tier 1: Data Wrangling & Database Schema Creation

**Objective:** Clean raw player, match, and session CSV files ( `raw_players.csv` , `raw_matches.csv` , `raw_sessions.csv` ), standardize null values, and structure them into a normalized SQLite relational database.

- Loaded raw datasets using `pandas`.
- Cleaned missing values in match data using `matches['outcome'].fillna('Draw')`.
- Established a persistent connection to `gaming_data.db` via `sqlite3.connect()`.
- Saved cleaned relational tables ( `players` , `matches` , `sessions` ) into SQLite using `df.to_sql(..., if_exists='replace')`.

- Validated data insertion by executing SQL queries ( `SELECT * FROM players LIMIT 5` ) and printing confirmation outputs.

## Tier 2: Exploratory Data Analysis & Visualizations

**Objective:** Extract joined relational data via SQL and construct visual representations to compare server ping latency and match outcome proportions across regional clusters.

- Built a 3-way SQL `JOIN` query combining `players`, `matches`, and `sessions` on `player_id`.
- Plot 1 ( `ping_distribution.png` ):** Generated a categorical boxplot ( `sns.boxplot` ) mapping `region` against `ping_ms`. Showcased median latencies, interquartile ranges, and outlier markers.
- Plot 2 ( `match_outcomes.png` ):** Generated a grouped count plot ( `sns.countplot` ) mapping `region` against match outcomes ( `Win`, `Loss`, `Forfeit` ) using the `viridis` color palette.
- Saved both charts as high-resolution PNG assets in the project root directory.

## Tier 3: Inferential Statistical Testing (ANOVA)

**Objective:** Perform a hypothesis test to determine if ping latency variances between regions are statistically significant.

### Hypotheses Formulation

- Null Hypothesis ( $H_0$ ):**  $\mu_{APAC} = \mu_{LATAM} = \mu_{EU}$  (Mean ping latency is equal across all regions).
- Alternative Hypothesis ( $H_1$ ):** At least one region has a mean ping latency different from the others.

**Actions & Execution:** Extracted ping values grouped by region array structures and executed SciPy's One-Way ANOVA test ( `stats.f_oneway(*regions)` ).

| METRIC                              | VALUE                  | INTERPRETATION                                                     |
|-------------------------------------|------------------------|--------------------------------------------------------------------|
| F-Statistic                         | 2.8991                 | Ratio of variance between region means to variance within regions. |
| P-Value                             | 5.6211e-02<br>(0.0562) | Probability of observing test results given $H_0$ is true.         |
| Significance Threshold ( $\alpha$ ) | 0.05                   | Standard 5% significance level.                                    |

**Statistical Conclusion:** Since the p-value ( 0.0562 ) is greater than the standard significance threshold ( $\alpha = 0.05$ ), we **fail to reject the null hypothesis**. There is no statistically significant difference in server ping latency across regions at the 5% significance level.

## Tier 4: Documentation & Repository Packaging

**Objective:** Format and package the complete project into a professional GitHub repository with a detailed markdown document.

- Created a comprehensive `README.md` file detailing project goals, installation guides, dataset breakdowns, visual insights, and statistical inferences.
- Embedded visual charts ( `ping_distribution.png`, `match_outcomes.png` ) directly into the markdown documentation.
- Verified repository file structures and tracked files using `.gitignore`.

- Completed final Git commits and pushed all assets to remote repository hosting on GitHub ( `master` branch).

## ⚠ Challenges, Errors & Troubleshooting Log

Throughout the project execution, several technical errors were encountered and systematically resolved:

1. `ModuleNotFoundError: No module named 'scipy'`
  - Cause: The 'scipy' module was not installed in the Python environment when Tier 3 code was executed.
  - Solution: Installed the library using 'pip install scipy' inside the terminal environment before re-running the cell.
2. Visual Studio Code Popup: "Running cells with 'Python 3.14.7' requires the ipykernel package"
  - Cause: VS Code defaulted to the global base Python environment instead of the project-specific virtual environment (.venv).
  - Solution: Manually switched the Jupyter notebook kernel back to '.venv\Scripts\python.exe' via the "Select Kernel" menu.
3. Git Output: "nothing to commit, working tree clean"
  - Cause: Attempted to run 'git commit' before saving modified notebook files (wrangling.ipynb).
  - Solution: Used Ctrl + S to save all open files before executing 'git add .' and 'git commit'.
4. Python Code Pasted directly into Terminal Prompt (>> import sqlite3...)
  - Cause: Code meant for Jupyter notebook cells was accidentally pasted into the active terminal prompt while a process was running.
  - Solution: Issued Ctrl + C in the terminal to cancel execution, cleared the terminal, and pasted the snippet into the notebook editor cell instead.

## 📌 Final Deliverable Verification Table

| FILE / ASSET NAME                  | DESCRIPTION / PURPOSE                                     | TIER   | STATUS ON GITHUB  |
|------------------------------------|-----------------------------------------------------------|--------|-------------------|
| <code>gaming_data.db</code>        | SQLite relational database containing cleaned tables      | Tier 1 | ✅ Synced & Pushed |
| <code>analysis.sql</code>          | SQL script queries                                        | Tier 1 | ✅ Synced & Pushed |
| <code>ping_distribution.png</code> | Boxplot showing regional server latency spread            | Tier 2 | ✅ Synced & Pushed |
| <code>match_outcomes.png</code>    | Grouped bar chart showcasing match outcome counts         | Tier 2 | ✅ Synced & Pushed |
| <code>wrangling.ipynb</code>       | Master notebook containing cleaning, visuals & ANOVA code | Tier 3 | ✅ Synced & Pushed |
| <code>README.md</code>             | Executive documentation and setup instructions            | Tier 4 | ✅ Synced & Pushed |